

CS 170 Programming Quiz 7



For **PQ07 - Processing Strings with Loops**, you'll write two methods that use loops. This exam covers Chapter 6 in your textbook. Of course, you'll have to know the material in previous chapters to know how to write methods and know how to make decisions. For this exam you'll need to know:

- How to correctly define the syntax of a method, given the specification.
- How to use the for loop to write counter-controlled loops that process Strings
- How to use the while loop to write loops using sentinels and limit loops

As on the previous exam, each of the two methods will be tested for syntax correctness (name, public, return type and parameters) and for correctness. Work step-by-step so that you get partial credit. You can find more problems to practice with on Codingbat and on Practicit. You'll find the links on Canvas.

1 THE WORD ENDS PROBLEM

Given a string and a non-empty word string, return a string made of each char just before and just after every appearance of the word in the string. Ignore cases where there is no char before or after the word, and a char may be included twice if it is between two words. Do not do any printing.

- for input of "abcXY123XYijk", "XY" → "c13i"
- for input of "XY123XY", "XY" → "13"
- for input of "XY1XY", "XY" → "11"

2 THE PLUS OUT PROBLEM

Given a string and a non-empty word string, return a version of the original string where all chars have been replaced by pluses ("+"), except for appearances of the word string which are preserved unchanged.

- for input of "12xy34", "xy" → "++xy++"
- for input of "12xy34", "1" → "1++++"
- for input of "12xy34xyabcxy", "xy" → "++xy++xy+++xy"

3 THE STAR OUT PROBLEM

Return a version of the given string, where for every star (*) in the input string the star and the chars immediately to its left and right are gone. So "ab*cd" yields "ad" and "ab**cd" also yields "ad".

- for input of "ab*cd" → "ad"
- for input of "ab**cd" → "ad"
- for input of "sm*eilly" → "silly"

4 THE NIK NAK PROBLEM

Look for patterns like "nik" and "nak" in the input string—in other words a substring, length-3, starting with 'n' and ending with 'k'. Return a string where for all such words, the middle letter is gone, so "nikXnak" yields "nkXnk".

- for input of "nikXnak" → "nkXzp"
- for input of "noknok" → "nknk"
- for input of "nnnoknok" → "nnnknk"

5 THE SANDWICH PROBLEM

A sandwich is two pieces of bread with something in between. Return the string that is between the first and last appearance of "bread" in the given string, or set it to the empty string "" if there are not two pieces of bread.

- for input of "breadjambread" → "jam"
- for input of "xxbreadjambreadyy" → "jam"
- for input of "xxbreadyy" → ""

6 THE REPEAT SEPARATOR PROBLEM

Given as input two strings, word and a separator, return a big string made of count occurrences of the word, separated by the separator string.

- for input of "Word", "X", 3 → "WordXWordXWord"
- for input of "This", "And", 2 → "ThisAndThis"
- for input of "This", "And", 1 → "This"

7 THE COUNT WORD PROBLEM

Write a method that takes two parameters: a String str, and a String word. Return the number of times that word appears inside str. The word may overlap in the String str, like "banana" and "ana" below.

- countWord("BobBob", "ob") returns 2
- countWord("BobBob", "BobB") returns 1
- countWord("banana", "ana") returns 2 // notice overlap

8 THE REPEAT FRONT PROBLEM

Given as input a string and an int n, return a string made of the first n characters of the string, followed by the first n-1 characters of the string, and so on. You may assume that n is between 0 and the length of the string, inclusive (i.e. $n \geq 0$ and $n \leq \text{str.length}()$).

- for input of "Chocolate", 4 → "ChocChoChC"
- for input of "Chocolate", 3 → "ChoChC"
- for input of "Ice Cream", 2 → "IcI"

9 THE REPEAT END PROBLEM

Given as input string and an int N, set result a string made of N repetitions of the last N characters of the string. You may assume that N is between 0 and the length of the string, inclusive.

- for input of "Hello", 3 → "llollollo"
- for input of "Hello", 2 → "lolo"
- for input of "Hello", 1 → "o"

10 THE MIX STRING PROBLEM

Given two input strings, A and B, create a bigger string made of the first char of A, the first char of B, the second char of A, the second char of B, and so on. Any leftover chars go at the end of the result.

- for input of "abc", "xyz" → "axbycz"
- for input of "Hi", "There" → "HTihere"
- for input of "xxxx", "There" → "xTxhxexre"

11 THE COUNT CODE PROBLEM

Return the number of times that the string "code" appears anywhere in the input given string, except we'll accept any letter for the 'd', so "cope" and "cooe" count.

- for input of "aaacodebbb" → 1
- for input of "codexxcode" → 2
- for input of "cozexxcope" → 2

12 THE COUNT DIGITS PROBLEM

Return the total number of "digit" characters that appear in the string. You must use the relational operators to determine if a character is a digit. You may not use the Character classification functions.

- countDigits("xxHxix") returns 0
- countDigits("abxxxc7") returns 1
- countDigits("7ab29c") returns 3

13 THE FRONT TIMES PROBLEM

Given an input string and a non-negative int n, we'll say that the front of the string is the first 3 chars, or whatever is there if the string is less than length 3. Return n copies of the front;

- for input of "Chocolate", 2 → "ChoCho"
- for input of "Chocolate", 3 → "ChoChoCho"
- for input of "Abc", 3 → "AbcAbcAbc"

14 THE STRING YAK PROBLEM

Suppose the string "yak" is unlucky. Given a string, return a version where all the "yak" are removed, but the "a" can be any char. The "yak" strings will not overlap.

- for input of "yakkak" → "pak"
- for input of "pakyak" → "pak"
- for input of "yak123ya" → "123ya"

15 THE EVERY NTH PROBLEM

Write a method that takes two parameters: a non-empty String str, and an int n. Return the String made starting with char 0, and then every nth char of the String. So if n is 3, use char 0, 3, 6, ... and so on. You can be sure that n is 1 or more.

- everyNth("Miracle", 2) returns "Mrce"
- everyNth("abcdefg", 2) returns "aceg"
- everyNth("abcdefg", 3) returns "adg"

16THE STRING BITS PROBLEM

Given a string, return a new string made of every other char starting with the first, so "Hello" yields "Hlo".

- for input of "Hello" → "Hlo"
- for input of "Hi" → "H"
- for input of "Heeololeo" → "Hello"

17THE COUNT LETTERS PROBLEM

Return the total number of "letter" characters (alphabetical, case insensitive) that appear in the string. You must use the relational operators to determine if a character is a letter. You may not use the Character classification functions.

- countLetters("7 52") returns 0
- countLetters("abxx xc7") returns 6
- countLetters("7 ab2 9c") returns 3

18THE STRING SPLOSION PROBLEM

Given a non-empty string like "Code" return a string like "CCoCodCode".

- for input of "Code" → "CCoCodCode"
- for input of "abc" → "aababc"
- for input of "ab" → "aab"

19 THE STRING X PROBLEM

Given a string, return a version where all the "x" have been removed. Except an "x" at the very start or end should not be removed.

- for input of "xxHxix" → "xHix"
- for input of "abxxxcd" → "abcd"
- for input of "xabxxcdx" → "xabc dx"

20THE GOOD NUMBERS PROBLEM

Given a String, return a new String containing only the digit characters from the String. You must use the relational operators to determine if a character is a digit; you cannot use the Character classification methods.

- goodNumbers("ab1c235") returns "1235"
- goodNumbers("\$2,12.50") returns "1250"
- goodNumbers("Heeololeo") returns ""

21 THE STRING MATCH PROBLEM

Given 2 strings, a and b, return the number of the positions where they contain the same length 2 substring. So "xxcaazz" and "xxbaaz" yields 3, since the "xx", "aa", and "az" substrings appear in the same place in both strings.

- for input of "xxcaazz", "xxbaaz" → 3
- for input of "abc", "abc" → 2
- for input of "abc", "axc" → 0

22 THE COUNT CHAR PROBLEM

Given a String str, and a char c return the number of times that c appears inside str.

- countChar("ABCDEFG", 'a') returns 0
- countChar("xxxx", 'x') returns 4
- countChar("aAaA", 'a') returns 2

23 THE LAST TWO PROBLEM

Given a string, return the count of the number of times that a substring length 2 appears in the string and also as the last 2 chars of the string, so "hixxhi" yields 1 (we won't count the end substring).

- for input of "hixxhi" → 1
- for input of "xaxxaxaxx" → 1
- for input of "axxxaaxx" → 2

24 THE COUNT NEIGHBOR PROBLEM

Given a String str, and a char c return the number of times that either the character before it (in the alphabet) or the character after it appears inside str.

- countNeighbor("ABCDEFG", 'a') returns 1 // the 'b'
- countNeighbor("xxxx", 'x') returns 0
- countNeighbor("abcd", 'b') returns 2 // 'a' and 'b'

25 THE COUNT YZ PROBLEM

Given an input string, count the number of words ending in 'y' or 'z' -so the 'y' in "heavy" and the 'z' in "fez" count, but not the 'y' in "yellow". Make sure that your comparison is not case sensitive. We'll say that a y or z is at the end of a word if there is not an alphabetic letter immediately following it.

- for input of "fez day" → 2
- for input of "day fez" → 2
- for input of "day fyzz" → 2

26 THE WITHOUT STRING PROBLEM

Given two input strings, base and remove, return a version of the base string where all instances of the remove string have been removed. This is not case sensitive. You may assume that the remove string is length 1 or more. Remove only non-overlapping instances, so with "xxx" removing "xx" leaves "x".

- for input of "Hello there", "llo" → "He there"
- for input of "Hello there", "e" → "Hllo thr"
- for input of "Hello there", "x" → "Hello there"

27 THE WITHOUT SPACE PROBLEM

Given an input string, return a new string with all whitespace characters removed. For this problem, we'll only consider space, tab and newline characters. You cannot use the Character classification functions for this problem.

- `removeWhitespace("How now?")` returns "Hownow?"
- `removeWhitespace("\nWha\t's this?")` returns "Wha'sthis?"
- `removeWhitespace(" \t\n ")` returns ""

28 THE COUNT LOWER PROBLEM

Return the total number of lower-case letter characters (alphabetical) that appear in the string. You must use the relational operators to determine if a character is a lower-case letter. You may not use the Character classification functions.

- `countLower("7 52")` returns 0
- `countLower("abxX xC7")` returns 4
- `countLower("7Aab2 9c")` returns 3

29 THE COUNT TRIPLE PROBLEM

We'll say that a "triple" in a string is a char appearing three times in a row. Return the number of triples in the given string. The triples may overlap.

- for input of "abcXXXabc" → 1
- for input of "xxxabyyyycd" → 3
- for input of "a" → 0

30 THE SUM DIGITS PROBLEM

Given a string, return the sum of the digits 0-9 that appear in the string, ignoring all other characters. Return 0 if there are no digits in the string.

- for input of "aa1bc2d3" → 6

- for input of "aa11b33" → 8
- for input of "Chocolate" → 0

31 THE COUNT UPPER PROBLEM

Return the total number of upper-case letter characters (alphabetical) that appear in the string. You must use the relational operators to determine if a character is an upper-case letter. You may not use the Character classification functions.

- countUpper("7 52") returns 0
- countUpper("abxX xC7") returns 2
- countUpper("7Aab2 9c") returns 1

32 THE SAME ENDS PROBLEM

Given a string, return the longest substring that appears at both the beginning and end of the string without overlapping. For example, given the input "abXab", then result "ab".

- for input of "abXYab" → "ab"
- for input of "xx" → "x"
- for input of "xxx" → "x"

33 THE MIRROR ENDS PROBLEM

Given a string, look for a mirror image (backwards) string at both the beginning and end of the given string. In other words, zero or more characters at the very beginning of the given string, and at the very end of the string in reverse order (possibly overlapping). For example, the string "abXYZba" has the mirror end "ab".

- for input of "abXYZba" → "ab"
- for input of "abca" → "a"
- for input of "aba" → "aba"

34 THE MAX BLOCK PROBLEM

Given an input string, return the length of the largest "block" in the string. A block is a run of adjacent chars that are the same.

- for input of "hoopla" → 2
- for input of "abbCCCddBBBxx" → 3
- for input of "" → 0

35 THE REVERSE CASE PROBLEM

Given an input string, return a string containing the same characters as the original, but where the case of each character is reversed; characters that were originally lower case should be upper and vice-versa. Remember that you can convert a character from uppercase to lowercase by adding 'a' - 'A', and convert a character from lowercase to uppercase by subtracting 'a' - 'A'. Of course you need to make sure you actually have an upper or lowercase character before you apply this.

- `reverseCase("7 52")` returns `"7 52"`
- `reverseCase("abxX xC7")` returns `"ABXx Xc7"`
- `reverseCase("7Aab2 9c")` returns `"7aAB2 9C"`

36 THE NOT REPLACE PROBLEM

Given an input string, return a string where every appearance of the lowercase word "is" has been replaced with "is not". The word "is" should not be immediately preceded or followed by a letter -- so for example the "is" in "this" does not count.

- for input of `"is test"` → `"is not test"`
- for input of `"is-is"` → `"is not-is not"`
- for input of `"This is right"` → `"This is not right"`